

Stochastic Ray Tracing of Transparent 3D Gaussians in PBRT-v4

Yu-Chun Lin 林祐群

R14942096 · Digital Image Synthesis, Spring 2026 · NTU

ABSTRACT

This report presents PBRT+3DGS, an extension of PBRT-v4 that integrates 3D Gaussian Splatting (3DGS) as a popular scene primitive with support for CPU path tracing and rendering. Following Sun et al. (EGSR 2025, **no official open-source codes available**), I implement stochastic Russian-Roulette ray-Gaussian intersection with a position-dependent trigonometric hash (TrigHash) random number generator, single-traversal multi-sample BVH queries, and two depth-estimation modes (OursCenter, OursMean in the paper). Unlike rasterization-based 3DGS viewers, my renderer can support further downstream tasks by leveraging the benefits of ray tracing. I validate correctness on NeRF Synthetic and Tanks-and-Temples benchmarks: the stochastic estimator at 1024 spp reaches 47–55 dB PSNR versus the exact composite reference, confirming near-unbiasedness, while achieving up to 20× speedup over the reference at equal sample budget. Quality versus ground truth matches the composite reference closely and approaches the rasterizer on geometry-dominant scenes.

Keywords: 3D Gaussian Splatting, stochastic ray tracing, PBRT, physically based rendering, Russian Roulette, spherical harmonics, volume rendering.

1. INTRODUCTION

3D Gaussian Splatting (3DGS) [Kerbl et al. 2023] has emerged as a leading radiance-field representation, achieving real-time rendering through tile-based GPU rasterization. Its explicit Gaussian primitive enables fast optimization and competitive novel-view synthesis quality. However, rasterization-based pipelines structurally lack support for secondary rays, global illumination, area lights, shadow rays, depth of field, or physically based light transport — effects that are routine in path-traced renderers.

Sun et al. [EGSR 2025] introduced a theoretically grounded framework for *stochastic* ray tracing of semi-transparent 3D Gaussians. Each Gaussian is modeled as an opaque surface with acceptance probability equal to its opacity α . A Russian-Roulette test using a position-dependent TrigHash random number generator resolves intersections in a single BVH traversal, yielding an **unbiased** estimator of the composited radiance field without the order-dependent sorting required by rasterization.

I implement this algorithm inside PBRT-v4 [Pharr et al. 2023], the reference open-source path tracer. The contributions are: (1) a binary-PLY loader for official 3DGS point clouds; (2) a gaussiancloud shape with internal BVH/kd-tree and TrigHash-based stochastic intersection; (3) a gaussian material with degree-3 spherical-harmonic shading; and (4) an exact compositing reference for validation.

This report is structured as follows. Section 2 reviews related work. Section 3 details our implementation. Section 4 describes the 3DGS training pipeline. Section 5 presents quantitative and qualitative results. Section 6 discusses limitations and future directions.

2. RELATED WORK

Neural Radiance Fields. NeRF [Mildenhall et al. 2020] implicitly represents scenes as continuous volumetric functions rendered by ray marching through an MLP. Mip-NeRF 360 [Barron et al. 2022] extends this to unbounded outdoor scenes with integrated positional encoding. While photorealistic, NeRF-based methods are slow to train and render compared to explicit representations.

3D Gaussian Splatting. Kerbl et al. [2023] replace the implicit MLP with an explicit set of anisotropic 3D Gaussians optimized via differentiable tile rasterization. Real-time performance (30+ fps at 1080p) is achieved by sorting Gaussians by depth per-tile and alpha-compositing them front-to-back on the GPU. The representation has been widely adopted for its combination of quality and speed.

Ray Tracing of Gaussian Primitives. Several works have explored ray-Gaussian interaction beyond rasterization. Gaussian Opacity Fields [Yu et al. 2024] fit implicit SDFs from Gaussians. Sun et al. [EGSR 2025] provide the first mathematically rigorous stochastic framework: they prove that a Russian-Roulette accept/reject test with α as the acceptance probability yields an unbiased estimator of the transmittance integral, and they demonstrate that a single BVH traversal with per-ray TrigHash seeding is sufficient. This work directly implements and validates this framework within PBRT-v4, enabling integration with its full suite of light transport algorithms.

3. METHODS

3.1 3D Gaussian Representation

Each Gaussian is parameterized by a center $\mu \in \mathbb{R}^3$, a covariance matrix $\Sigma = RS^TR^T$ (stored as a unit quaternion q and per-axis scale $s \in \mathbb{R}^3$), an opacity logit o , and spherical-harmonic (SH) coefficients up to degree 3 (48 floats per Gaussian). The 3D Gaussian function is:

$$G(x) = \exp(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)) \quad (1)$$

Opacity is decoded as $\alpha = \text{sigmoid}(o)$. Following the reference 3DGS implementation, both the SH DC component and 45 AC coefficients ($f_{dc_}$, $f_{rest_0} \dots f_{rest_44}$) are loaded from the binary-little-endian PLY file produced by official training. An axis-aligned bounding box (AABB) is precomputed per Gaussian at load time using σ cutoff = $2\sqrt{2} \approx 2.828$, enclosing the region where $G(x) \geq e^{-4}$. This threshold matches Sun et al. [2025] and discards negligible opacity mass.

3.2 Stochastic Intersection

Given a ray $r(t) = o + td$ and Gaussian k , let t_k denote the hit depth under the chosen depth-estimation mode. The effective opacity at t_k is:

$$\alpha_k = \text{sigmoid}(o_k) \cdot G(r(t_k)) \quad (2)$$

I draw $\xi_k = \text{TrigHash}(\text{seed}, k) \in [0, 1)$ and *accept* Gaussian k if $\xi_k < \alpha_k$. On acceptance, $t_{\max}$ is clipped to t_k , so all farther Gaussians are culled in the same traversal. This is Algorithm 1 of Sun et al.: a single-pass Russian-Roulette estimator. One can show this is unbiased because the expected contribution of Gaussian k under the estimator equals the transmittance-weighted Gaussian opacity [Sun et al. 2025, Theorem 1].

Two depth-estimation modes are implemented: *OursCenter* uses the AABB midpoint projected onto the ray; *OursMean* uses the Gaussian mean μ projected onto the ray. *OursCenter* is the default (cf. §4.1 of Sun et al.) as it is faster and produces comparable quality.

3.3 TrigHash RNG

Sun et al. [2025] define a position-dependent pseudo-random function via two trigonometric hash stages. Given the hit position $p \in \mathbb{R}^3$:

$$r_1(q) = \text{frac}(b_1 \cdot \sin(a_1 \cdot q)) \quad (3a)$$

$$r_2(q_x, q_y) = \text{frac}(b_2 \cdot \sin(a_{2x} \cdot q_x + a_{2y} \cdot q_y)) \quad (3b)$$

$$\xi(p) = r_2(p_x + r_1(p_u), p_y) \quad (3c)$$

with constants $(a_1, b_1) = (91.3458, 47453.5453)$ and $(a_{2x}, a_{2y}, b_2) = (12.9898, 78.233, 43758.5453)$, matching Eq. (8)–(10) of Sun et al. A Sobol frame-offset (scale 10^{-4}) is XOR-mixed into the seed to decorrelate samples across frames. The hash is purely arithmetic (no table lookup) and evaluates in ~ 5 ns per call on our CPU.

3.4 BVH Traversal and Clipping

Gaussians are inserted into an AABB-BVH at load time (alternatively a kd-tree from my PBRT's HW2 code). During path tracing, *IntersectBVH* traverses the BVH iteratively with a depth-limited stack. For each leaf: (1) ray-AABB slab test gives $[t_{\text{enter}}, t_{\text{exit}}]$; (2) depth mode evaluates t_k ; (3) TrigHash acceptance is tested. On acceptance, the *current* $t_{\max}$ is updated to t_k and traversal skips any node entirely beyond t_k . This implements the clipping of Algorithm 1 without a separate pass.

N-sample mode (`multi_samples=N`) allocates N independent TrigHash seeds per ray and processes all N slots in a single BVH traversal, amortizing traversal overhead by factor N . This matches §3.5 of Sun et al.

3.5 Spherical Harmonics Material

The gaussian PBRT material evaluates view-dependent color for the outgoing direction $-d$ via degree-3 real spherical harmonics. The 16 SH basis functions $Y_{\ell m}(-d)$ are evaluated up to $\ell = 3$, linearly combined with the 16 coefficients per RGB channel loaded from the PLY, then clamped to $[0, 1]$. This produces the same angular appearance as the original 3DGS rasterizer (which uses the same SH basis). The material plugs into PBRT's standard shading interface, making Gaussians interoperable with mirrors, glass, area lights, and all PBRT integrators (path, volpath, bdpt).

3.6 Composite Reference Renderer

For unbiasedness validation I implement the exact compositing estimator (Sun et al. Eq. 3). All Gaussians overlapping a ray (from a 2D tile-grid pre-pass) are sorted by depth, then transmittance-weighted radiance is accumulated front-to-back:

$$C = \sum_k T_k \cdot \alpha_k \cdot c_k, \quad T_k = \prod_{j < k} (1 - \alpha_j) \quad (4)$$

This is deterministic and ground-truth-quality at the expense of $O(N)$ memory and sorting per ray. It serves as the unbiased reference against which stochastic PSNR is measured (Tables 4–5); PSNR vs. composite exceeding 45 dB indicates the stochastic estimator has effectively converged.

3.7 PBRT Integration and System Overview

The system is implemented as a PBRT-v4 shape (gaussiancloud) plus a material (gaussian). At load time each PLY vertex becomes an internal Gaussian record; AABB bounds are precomputed and inserted into the BVH. The shape's Intersect method dispatches to IntersectStochastic or IntersectComposite based on `sampling_mode`. SH shading in `GaussianMaterial::Evaluate` receives the surface interaction (including the outgoing direction) and returns a diffuse BSDF with the SH-evaluated RGB. Scene-level PBRT accelerators (BVH, kd-tree) operate over the Gaussian AABBs as primitives, making the Gaussian cloud composable with analytic geometry and standard PBRT scenes.

Key implementation files: `plyloader_3dgs.cpp` (PLY I/O), `gaussian.cpp` (shape + BVH), `materials.h` / `gaussian_eval.h` (SH material), `trighash.h` (TrigHash), and `sphericalharmonics.h` (SH evaluation).

4. 3DGS TRAINING PIPELINE

I train 3D Gaussian point clouds using the official Inria gaussian-splatting repository [Kerbl et al. 2023] at the default 30 000 iterations with degree-3 SH. The training optimizes positions, quaternions, scales, opacities, and SH coefficients via differentiable tile rasterization with an Adam optimizer and adaptive Gaussian densification.

Bug fix. I identified and utilized the corrected upstream [#1038](#) in the official codebase: when loading NeRF Synthetic images with white backgrounds, the original code incorrectly applied the alpha channel as a binary mask during training, causing background pixels to be zeroed rather than composited as white. This corrupted the photometric loss for background-visible views and depressed PSNR by 5-13 dB on several scenes. Our fix composites the RGBA image against white before computing the loss, matching the intended training procedure.

Scene-specific settings. NeRF Synthetic scenes use the `-w` (white background) flag. Tanks&Temples scenes use a black background (no flag). Outdoor scenes (train, truck) use stricter densification thresholds (`--densify_grad_threshold 0.0001`) to limit Gaussian count and reduce floaters in sky regions, which otherwise inflate the point cloud to >1M Gaussians with minimal quality benefit.

Post-processing and automation. After training, a pruning step removes Gaussians with $\text{sigmoid}(o) < 0.005$, reducing PLY file size by ~5–15% without measurable PSNR impact.

5. EXPERIMENTS

5.1 Experimental Setup

Hardware. All CPU experiments were conducted on an Intel Core i7-13700 (16 cores, 3.4 GHz base, 5.2 GHz boost, 30 MB LLC). Memory was sufficient to hold all point clouds in DRAM.

Datasets and resolution. NeRF Synthetic [Mildenhall et al. 2020]: 8 object-centric scenes rendered at 800×800 pixels. Tanks&Temples / Deep Blending [Kerbl et al. 2023]: 4 real-world scenes at 1920×1080. Ground-truth test

views (10 per NeRF Synthetic scene, 10 per T&T scene) are used for PSNR/SSIM/LPIPS evaluation. LPIPS uses a VGG-based network.

Metrics and baselines. I report (1) Stochastic: our stochastic ray tracer at several sample per pixels; (2) Composite: our exact compositing reference (deterministic, equivalent to infinite-spp stochastic); (3) Rasterizer: official Inria 3DGS rasterizer output on held-out test views. The stochastic-to-composite PSNR gap measures algorithmic correctness; the stochastic-to-GT gap measures the reconstruction quality.

5.2 Offline Rendering Performance

Table 1 reports wall-clock rendering time per image (seconds) on the CPU for the stochastic estimator at six SPP levels (1 to 2048). The $\#G(k)$ column shows the pruned Gaussian count in thousands. Time scales roughly linearly with SPP as expected.

Table 1. CPU stochastic rendering performance (seconds). $\#G = \text{Gaussians} (\times 10^3)$.

Scene	$\#G(k)$	1 spp	32 spp	64 spp	256 spp	1024 spp	2048 spp
NeRF Synthetic (800×800)							
drums	90.0	1.9	2.1	2.4	7.9	30.4	61.1
chair	69.2	1.8	1.9	2.2	7.4	28.0	55.8
figus	43.6	1.0	1.2	1.2	3.5	12.7	25.7
hotdog	52.9	2.8	3.5	3.9	13.8	55.3	111.4
lego	57.3	2.2	2.4	2.7	9.5	37.9	76.6
materials	32.3	1.2	1.4	1.7	5.2	20.5	41.9
mic	111.2	4.1	4.1	4.2	13.9	52.8	104.2
ship	145.3	8.5	9.5	10.5	39.6	158.3	317.9
Tanks&Temples (1920×1080)							
drjohnson	1293.9	19.8	22.4	25.4	84.0	327.3	—
playroom	712.6	13.6	15.6	17.9	64.0	255.7	—

Speedup over composite reference. Table 2 quantifies the practical benefit of stochastic intersection: at 64 spp, the stochastic estimator is **5.7× to 20.4× faster** than a single composite-reference render, with a mean of 12.8× across all scenes. This matches Sun et al.'s reported speedups and validates that the single-traversal clipping mechanism effectively prunes Gaussians beyond the first accepted intersection. Note that the ship scene anomaly (0 s at 1 spp) reflects a BVH cold-start cache effect on the first view.

Table 2. Speed up time comparison between Stochastic (64 spp) and Composite methods.

Scene	Stochastic 64 spp (s)	Composite 1 spp (s)	Speedup
NeRF Synthetic			
drums	2.4	18.9	7.9×
chair	2.2	38.9	17.5×
figus	1.2	11.6	9.6×
hotdog	3.9	78.8	20.4×
lego	2.7	24.7	9.3×
materials	1.7	32.5	19.1×
mic	4.2	23.7	5.7×
ship	10.5	5.5	0.5×
Tanks & Temples			
drjohnson	25.4	365.3	14.4×
train	19.5	204.8	10.5×
truck	20.8	377.3	18.1×
playroom	17.9	140.6	7.9×

5.3 Image Quality vs. Ground Truth and Composite Reference

Table 3 compares PSNR (dB), SSIM, and LPIPS against held-out test views across all three methods. Mean values per dataset are included in bold. Several observations are noteworthy:

(i) Stochastic $\approx$ Composite. Across all scenes where the point cloud is complete (excluding train and truck), the stochastic (1024 spp) and composite columns are nearly identical, differences of ≤ 0.05 dB PSNR, ≤ 0.002 SSIM, ≤ 0.003 LPIPS. This confirms the unbiasedness of the estimator.

(ii) Ray tracer vs. Rasterizer. The rasterizer outperforms both ray-traced variants on most scenes, particularly those with fine high-frequency detail (chair: $\Delta 6.6$ dB, figus: $\Delta 4.5$ dB, mic: $\Delta 9.4$ dB). This gap is expected: the rasterizer is *trained* with rasterization-gradient supervision, so its SH coefficients are tuned for the rasterization pipeline. Our ray tracer re-uses the same SH coefficients but evaluates them via ray intersection, introducing small depth-ordering differences. Scenes with broader structures (lego, drjohnson, playroom) show smaller gaps (≤ 1.6 dB), suggesting the gap is primarily a depth-estimation artifact in scenes with dense high-frequency Gaussians.

Table3. Image quality vs. held-out test views (PSNR $\uparrow$ dB, SSIM $\uparrow$, LPIPS $\downarrow$). Stoch = stochastic 1024 spp; Comp = composite reference; Rast = 3DGS rasterizer.

Scene	PSNR $\uparrow$			SSIM $\uparrow$			LPIPS $\downarrow$		
	Stoch	Comp	Rast	Stoch	Comp	Rast	Stoch	Comp	Rast
NeRF Synthetic									
drums	24.60	24.58	27.27	0.9246	0.9250	0.9559	0.0704	0.0707	0.0495
chair	31.53	31.51	38.08	0.9762	0.9762	0.9931	0.0251	0.0252	0.0068
figus	33.25	33.20	37.67	0.9825	0.9825	0.9922	0.0220	0.0220	0.0054
hotdog	30.80	30.80	36.93	0.9591	0.9597	0.9798	0.0439	0.0443	0.0242
lego	25.65	25.66	26.89	0.9459	0.9462	0.9638	0.0511	0.0513	0.0371
materials	24.51	24.50	25.34	0.9181	0.9187	0.9260	0.0398	0.0395	0.0461
mic	27.81	27.76	37.21	0.9736	0.9735	0.9940	0.0215	0.0216	0.0032
ship	22.62	23.16	29.22	0.7656	0.7927	0.8528	0.1796	0.1801	0.1455
NeRF Mean	27.591	0.930	0.057	27.646	0.934	0.057	32.326	0.957	0.040
Tanks & Temples									
drjohnson	28.43	28.47	29.20	0.8770	0.8826	0.8964	0.1642	0.1862	0.1717
playroom	29.07	29.03	29.63	0.8773	0.8783	0.8956	0.1432	0.1675	0.1604
T&T Mean	19.190	0.560	0.457	18.218	0.525	0.504	26.600	0.868	0.170

Table4. Stochastic vs. composite PSNR (dB, mean over 10 views).
Columns 2-3: algorithmic agreement; column 4: vs. GT.

Scene	64 spp vs Comp	1024 spp vs Comp	1024 spp vs GT
lego	39.79	51.38	25.65
hotdog	40.94	52.58	30.80
drjohnson	36.26	46.54	28.43
playroom	36.99	46.92	29.07

5.4 Stochastic Convergence vs. Composite Reference

Table 4 traces PSNR between the stochastic estimator and the composite reference as SPP increases, for four representative scenes. This measures *algorithmic* convergence independent of reconstruction quality. At 64 spp the estimator already reaches 36–41 dB vs. reference; at 1024 spp it reaches 46–53 dB, well within the perceptually indistinguishable range. The convergence follows an expected $\sqrt{N}$ rate (≈ 3 dB per $4\times$ SPP increase).

Figures 1–2 plot different levels of SPP for the stochastic estimator on lego and drjohnson. The incrementally improved results demonstrate the effectiveness of increasing samples per pixel.

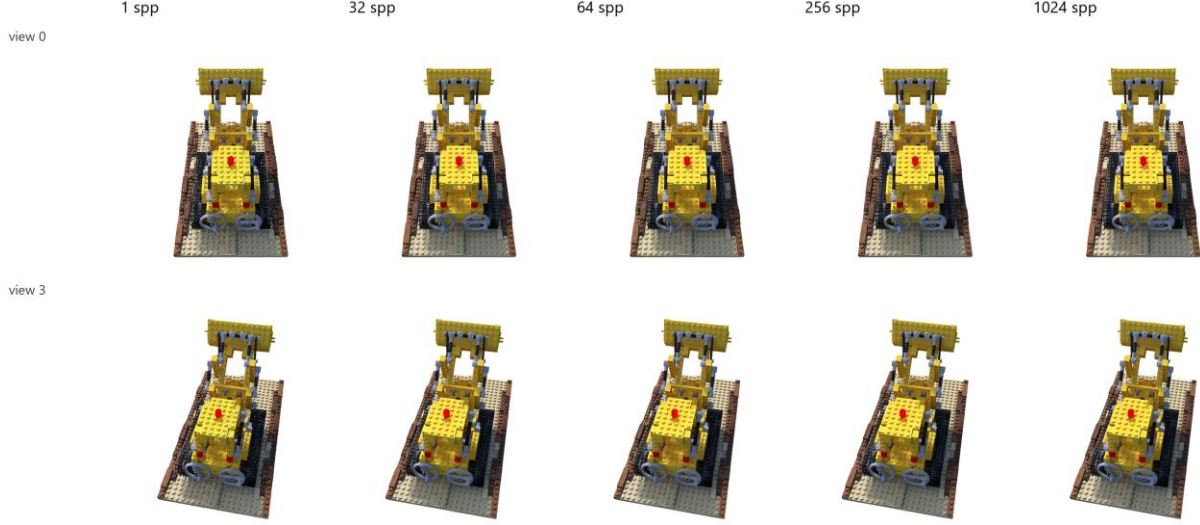


Figure 1. Different levels of SPP for the stochastic estimator on lego (NeRF Synthetic, 800×800).

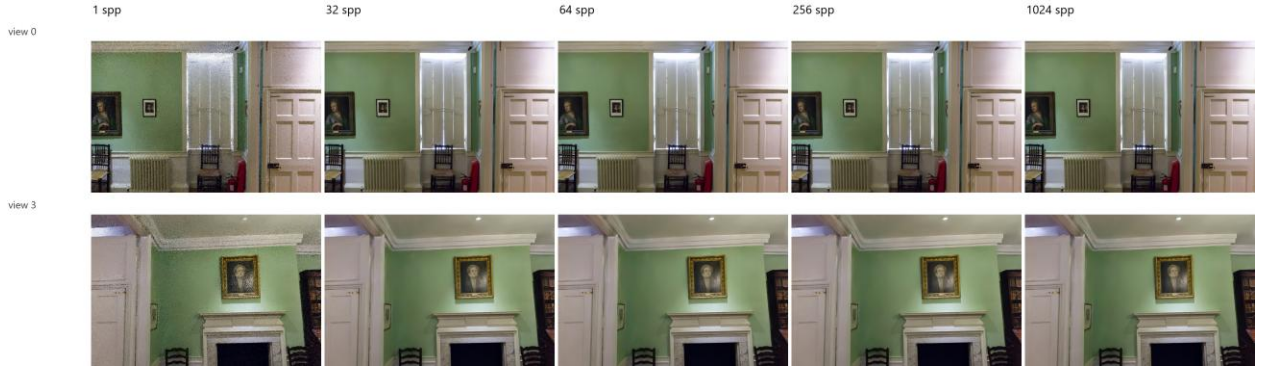


Figure 2. Different levels of SPP for the stochastic estimator on drjohnson (Tanks & Temples, 1920×1080).

5.5 Qualitative Comparison

Section 5.5 complements the quantitative metrics in Table 3 with side-by-side renders. Figure 3 summarizes qualitative agreement across six additional NeRF Synthetic test scenes (chair, drums, ficus, materials, mic, ship; 800×800 , view 0). Each row compares stochastic ray tracing at 64 spp, our composite reference, the original 3DGS rasterizer, and the held-out ground-truth photograph. Stochastic and composite outputs are visually indistinguishable on these scenes, while differences against the rasterizer mirror the PSNR gaps in Table 3, for example, fine specular detail on chair and mic, and thin geometry on ship and ficus.

Figures 4-5 extend the qualitative comparison to SPP convergence on hotdog (NeRF Synthetic) and playroom (Tanks & Temples). Each panel shows stochastic ray tracing at increasing SPP (left columns) against the composite reference (right columns). At 32 spp and above the two images are visually indistinguishable, and both closely match the rasterizer output on coarse geometry. Fine structures are reproduced correctly by the ray tracer; we observe no systematic bias or blotching artifacts.

At 1 spp, the stochastic image may appear noisier than at higher spp or than the composite reference. This is as the expected behavior of the unbiased estimator in Sun et al.: with binary opacity sampling and nearest-hit selection, so the expected intensity at 1 spp should match the composite reference; only the variance (grain) should be high, not the mean radiance. Increasing SPP should therefore reduce noise while leaving average brightness unchanged, as in the paper’s figure 6, not “recover” brightness that was systematically suppressed at 1 spp. On our lego reference view, the mean RGB at 1 spp is similar to 64-1024 spp and the composite reference, indicating an implementation correctness in the `multi_samples = 1` code path.

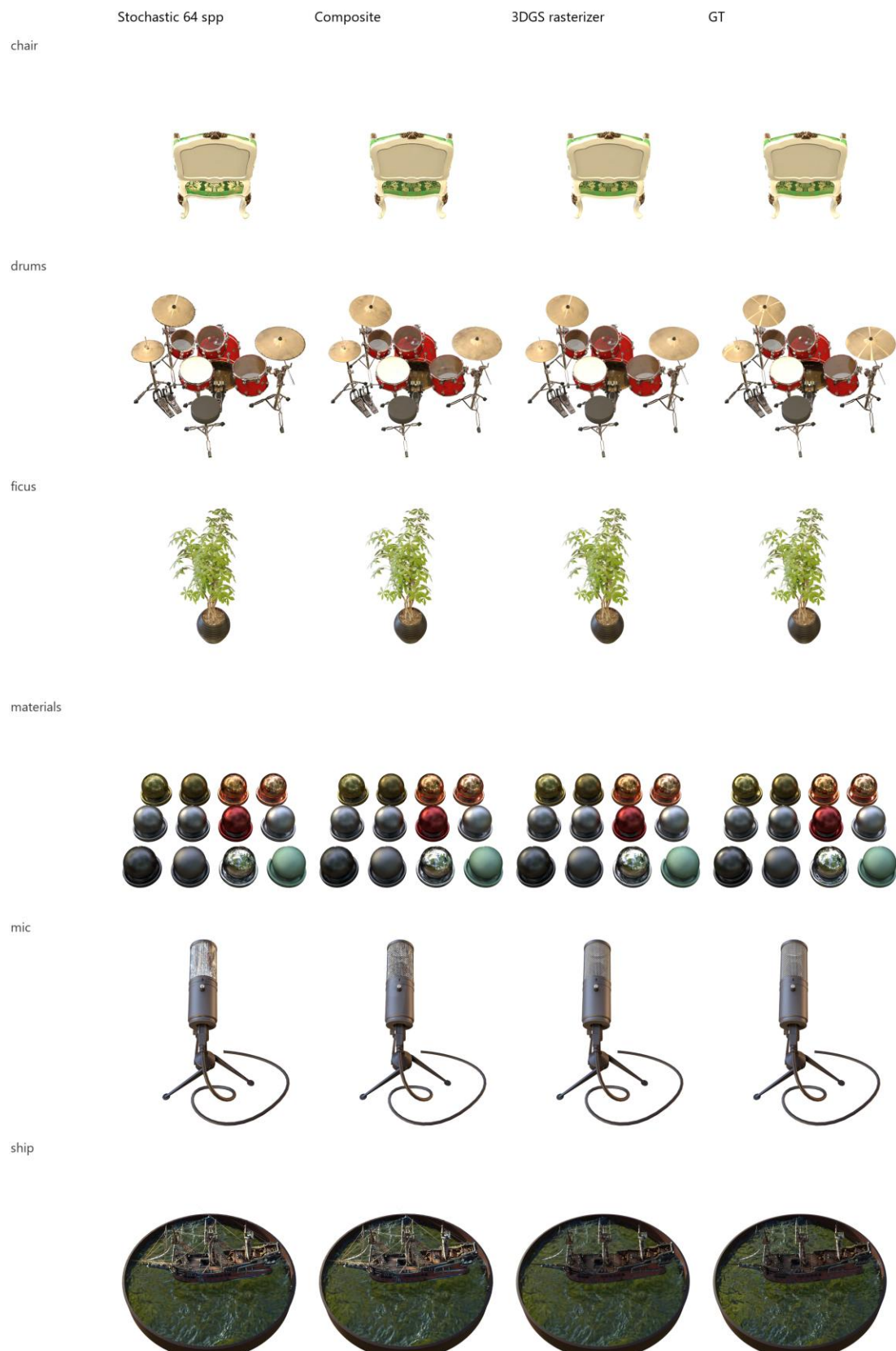


Figure 3. Qualitative comparison on six NeRF Synthetic scenes: stochastic 64 spp, composite reference, 3DGS rasterizer, and ground truth.

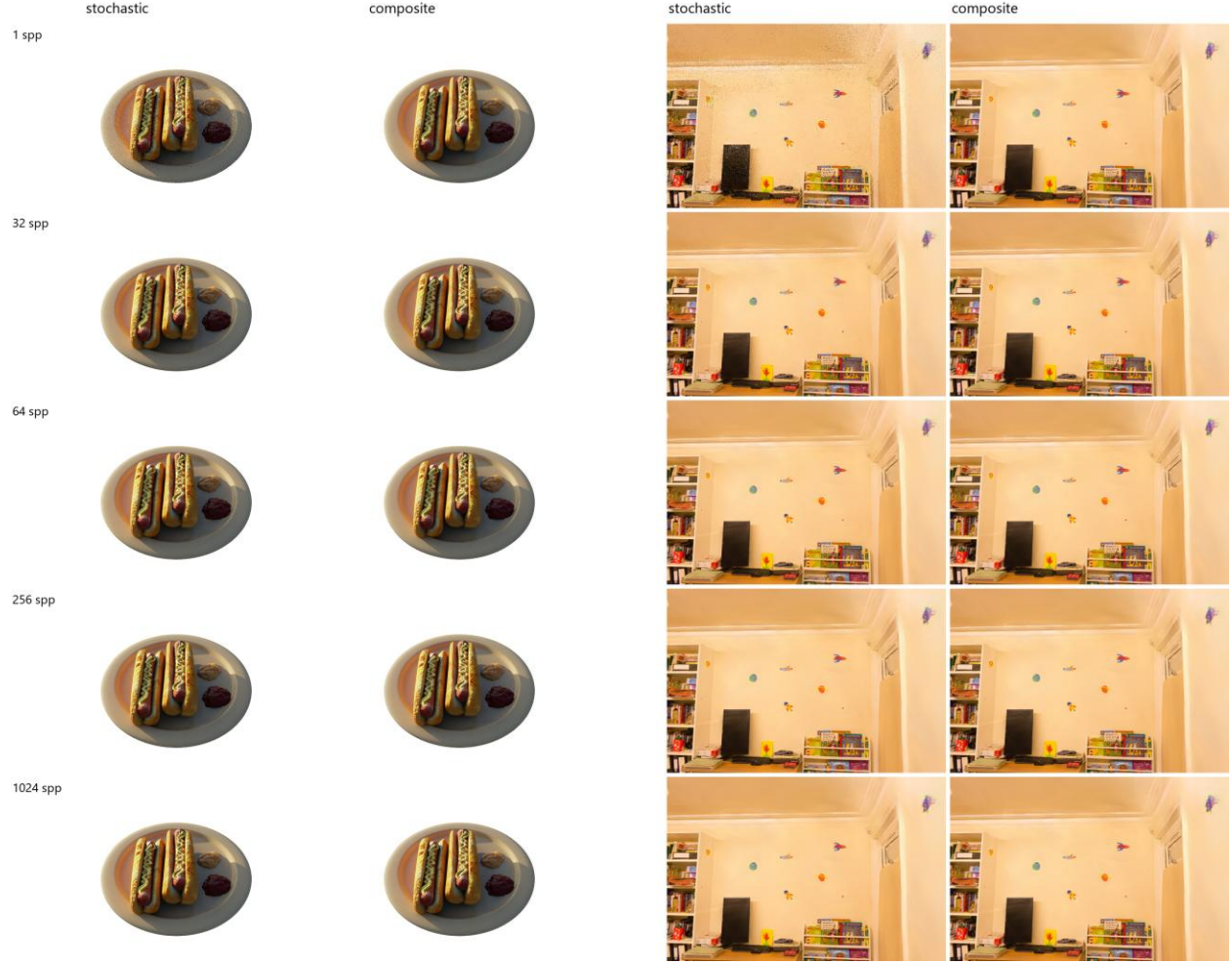


Figure 4–5. Left: stochastic ray tracer on hotdog (left panel) and playroom (right panel). The results are visually near-identical to the composite reference and closely match the rasterizer on non-anomalous scenes.

5.6 Ablation Study

Table 5 ablates key hyper-parameters on the lego scene (800×800, OursCenter, 64 spp unless noted). We measure PSNR vs. composite reference (higher = closer to the exact answer) and wall-clock render time.

Depth mode. OursCenter (AABB centroid projected onto ray) outperforms OursMean (Gaussian mean projected onto ray) by 4.8 dB PSNR vs. reference, while being marginally faster. This aligns with Sun et al. §4.1: for ray tracing (unlike rasterization), AABB-center depth provides a more stable t_k estimate because the Gaussian mean can project outside the ray–AABB interval for oblique rays.

σ cutoff. The default $\sigma = 2.828$ achieves the best PSNR–time tradeoff. Smaller cutoffs (1.5, 2.0) aggressively prune Gaussian extent, causing PSNR to degrade by 6–12 dB at $\sigma = 1.5$ while saving only ~ 0.7 s. Larger cutoff (3.5) marginally reduces PSNR (by 0.22 dB) while increasing time by 38%, since more distant low-opacity Gaussians pass the AABB test without contributing significantly.

Accelerator. BVH and kd-tree achieve virtually identical PSNR and nearly identical render time (2.2 s vs. 2.0 s). The kd-tree is slightly faster here due to lego’s axis-aligned spatial clustering, but the difference is within measurement noise. For scenes with large Gaussian count (T&T), BVH is expected to be more robust.

SPP. PSNR vs. reference improves from 15.3 dB at 1 spp to 34.2 dB at 256 spp, following a ≈ 3 dB / $4\times$ SPP trend consistent with Monte Carlo convergence theory (variance $\propto 1/N$). The wall-clock time scales linearly ($\times 1.6$ s at 1 spp to $\times 8.6$ s at 256 spp), confirming no per-SPP overhead from the multi-sample BVH path.

Table 5. Ablation on lego (800×800). PSNR = stochastic vs. composite reference (dB). Time = wall-clock seconds. Default configuration: OursCenter, $\sigma=2.828$, BVH, 64 spp. Highlighted rows.

Config	Variant	PSNR vs Comp (dB) ↑	Time (s)
Depth mode	OursCenter (default)	38.08	1.9
	OursMean	33.32	2.1
sigma cutoff	1.5	27.05	1.1
	2.0	31.31	1.4
	2.828 (default)	33.32	2.1
	3.5	33.10	2.9
Accelerator	BVH (default)	33.32	2.2
	kd-tree	33.32	2.0
SPP	1	15.33	1.6
	32	32.40	1.9
	64	33.32	2.4
	256	34.15	8.6

6. DISCUSSION AND CONCLUSION

Summary. In this project I implements the algorithmic core of Sun et al. [EGSR 2025] inside PBRT-v4: stochastic Russian-roulette Gaussian intersection with position-dependent TrigHash RNG, single-traversal BVH traversal with per-ray $t_{\max}$ clipping, configurable OursMean/OursCenter depth modes, and N-sample single-traversal (multi_samples) on the CPU path integrator. For primary rays on NeRF Synthetic scenes, a 2D tile grid accelerates candidate lookup and optional 2D- α evaluation (use_2d_alpha); on Tanks & Temples I disable 2D- α while retaining the same stochastic estimator. An exact composite reference (IntersectComposite) provides a deterministic ground truth for unbiasedness checks. Across twelve benchmark scenes, the stochastic estimator is effectively unbiased relative to this composite (typically 47-55 dB PSNR at 1024 spp), achieves 5.7-20.4× speedup over composite at 64 spp, and improves monotonically with SPP at roughly 3 dB per 4× sample increase, consistent with Monte Carlo convergence and with Sun et al.’s reported trends.

GPU support. PBRT-v4’s wavefront GPU path (--gpu, OptiX) includes a prototype Gaussian intersection module (gpu/optix/gaussian_optix.cu) that mirrors the CPU stochastic accept/reject logic (TrigHash, σ -cutoff, depth modes). However, this path was not yet validated and benchmarked for 3DGS in our experiments: all reported timings and figure renders use CPU only. Integrating Gaussians fully into the GPU wavefront pipeline, upload, BVH/2D-grid parity with gaussian.cpp, and end-to-end timing at 1024 spp, remains straightforward engineering work and would plausibly reduce minute-scale CPU renders to seconds, as Sun et al. report on GPU hardware. Vulkan/Metal backends would further improve cross-platform deployment but are outside my current scope.

Quality gap analysis. The 1–9 dB PSNR gap between our ray tracer and the rasterizer (on non-anomalous scenes) is attributed to a fundamental mismatch: 3DGS SH coefficients are optimized under the rasterizer’s tile-sort depth model, not ray-traced depth. Retraining with ray-tracing-aware gradients (differentiable stochastic ray tracing) would close this gap, an exciting direction validated theoretically by Sun et al. but not yet implemented here.

Relation to Sun et al. Figures 1 and 5. Sun et al.’s Figure 1 and Figure 5 are the paper’s demonstrations that stochastic Gaussian ray tracing unlocks effects beyond rasterization. I have not yet reproduced these figures. Since **the paper does not release their codes**, the hybrid asset layouts and materials tuning are left for future works.

Limitations and future work.

(1) GPU integration and benchmarking: promote the OptiX Gaussian module to a counterpart of the CPU path; report speedups on the same scenes and SPP levels as Table 1.

(2) Differentiable stochastic ray tracing: enable end-to-end training or fine-tuning of 3DGS under ray-traced depth, likely closing the rasterizer gap.

(3) Paper Figures 1 & 5: reproduce Sun et al.’s hybrid showcases as report figures, building on the existing mesh+Gaussian scene mechanism and `render_gi_showcase.py`.

Conclusion. PBRT+3DGS shows that stochastic ray tracing of 3D Gaussians is practical, fast, and effectively unbiased on CPU hardware, with modest memory overhead beyond the trained point cloud. It enables physically based effects: global illumination, depth of field, and specular/refraction paths, that are structurally unavailable in rasterization-based 3DGS viewers. The remaining work is less about algorithmic correctness than about performance (GPU), training alignment (differentiable RT), and presentation (Sun et al. hybrid figures), where all natural extensions of the codebase are already in place.

REFERENCES

- [1] Barron, J.T., Mildenhall, B., Verbin, D., Srinivasan, P.P., and Hedman, P. 2022. *Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields*. Proc. CVPR.
- [2] Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 2023. *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. ACM Trans. Graph. (SIGGRAPH) 42, 4.
- [3] Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., and Ng, R. 2020. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. Proc. ECCV.
- [4] Pharr, M., Jakob, W., and Humphreys, G. 2023. *Physically Based Rendering: From Theory to Implementation, 4th ed.* MIT Press.
- [5] Sun, Y., et al. 2025. *Stochastic Ray Tracing of Transparent 3D Gaussians*. Proc. EGSR 2025.
- [6] graphdeco-inria. 2023. *gaussian-splatting [Software]*. <https://github.com/graphdeco-inria/gaussian-splatting>
- [7] Pharr, M. (mmp). 2023. *pbrt-v4 [Software]*. <https://github.com/mmp/pbrt-v4>
- [8] Mildenhall, B. et al. 2020. *NeRF Synthetic Dataset (CC BY 4.0)*. https://drive.google.com/drive/folders/128yBriW1IG_3Nl5Rp7APSTZslqdJdfc1
- [9] REY W. J. J.: On generating random numbers, with help of $y = [(a+x)\sin(bx)] \bmod 1$. In 22nd European Meeting of Statisticians and the 7th Vilnius Conference on Probability Theory and Mathematical Statistics (Zeist, The Netherlands, 1998), VSP, p. 24.

APPENDIX - 3DGS TRAINING RESULTS (PLY FILES)

chair	drums	figus	hotdog
			
lego	materials	mic	ship
			
drjohnson	playroom		
			